

1. Student Attendance Analysis A college maintains the daily attendance details of its students in the form of a list containing student IDs. Some students may have attended multiple sessions on the same day. The administration wants to identify the longest continuous sequence of sessions in which no student ID is repeated. Develop a solution that determines the maximum length of such a sequence.

```
arr = list(map(int, input().split()))
seen = set()
left = 0
maximum = 0
for right in range(len(arr)):
    while arr[right] in seen:
        seen.remove(arr[left])
        left += 1
    seen.add(arr[right])
    length = right - left + 1
    maximum = max(maximum, length)
print(maximum)
```

```
1 2 3 1 4 5
5
```

Input	Output
1 2 3 1 4 5	5
1 2 3 4 5	5
1 1 1 1	1
1 2 3 2 4 5 6	5
5 4 3 2 1 2 3	5

2. Online Shopping Price Analysis An online shopping application stores the prices of products viewed by a customer during a browsing session. The customer wants to identify a continuous range of products that provides the maximum possible total discount value. Given the discount values, determine the maximum value that can be obtained from any continuous range.

```
arr = list(map(int, input().split()))
current = arr[0]
maximum = arr[0]
for i in range(1, len(arr)):
    current = max(arr[i], current + arr[i])
    maximum = max(maximum, current)
print(maximum)
```

```
-2 5 -1 6 -3
10
```

Input	Output
-2 5 -1 6 -3	10
1 2 3 4 5	15
-5 -2 -8 -1	-1

Input	Output
4 -1 2 1 -5 4	6
-2 3 -1 5 -6 4	7

3. Rainwater Collection System A city installs buildings of different heights along a straight road. During rainfall, water gets collected between taller buildings. The engineering team needs to calculate the total amount of water that can remain trapped after heavy rainfall based on the heights of the buildings.

```
heights=eval(input())
water=0
for i in range(len(heights)):
    left=max(heights[:i+1])
    right=max(heights[i:])
    water+=min(left,right)-heights[i]
print(water)
```

Test Case	Input	Expected Output
1	3 0 2 0 4	7
2	0 1 0 2 1 0 1 3 2 1 2 1	6
3	3 0 0 2 0 4	10
4	1 2 3 4 5	0
5	5 0 5	5

4. Employee Performance Analysis A company stores the monthly performance scores of an employee for several months. The scores may contain both positive and negative values depending on the employee's performance. Management wants to identify the continuous period during which the employee achieved the highest overall performance.

```
arr = list(map(int, input().split()))
current = arr[0]
maximum = arr[0]
for i in range(1, len(arr)):
    current = max(arr[i], current + arr[i])
    maximum = max(maximum, current)
print(maximum)
```

Test Case	Input	Expected Output
1	-2 5 -1 6 -3	10
2	1 2 3 4 5	15
3	-5 -2 -8 -1	-1
4	4 -1 2 1 -5 4	6
5	-2 3 -1 5 -6 4	7

5. Product Sales Analysis A retail company stores the daily sales quantity of a product for several consecutive days. Due to seasonal changes, some days may have negative adjustments. The company wants to identify the period that produced the highest multiplication of sales-related values. Develop a solution to determine this maximum product.

```

arr = list(map(int, input().split()))
current_max = arr[0]
current_min = arr[0]
maximum = arr[0]
for i in range(1, len(arr)):
    if arr[i] < 0:
        current_max, current_min = current_min, current_max
        current_max = max(arr[i], current_max * arr[i])
        current_min = min(arr[i], current_min * arr[i])
    maximum = max(maxim
print(maximum)

```

Test Case	Input	Expected Output
1	2 3 -2 4	6
2	-2 3 -4	24
3	-2 0 -1	0
4	2 3 -2 4 -1	48
5	-2 -3 -4	12

6. Customer Purchase History An e-commerce application stores the product IDs purchased by a customer in chronological order. The same product may appear multiple times. The system needs to determine the longest sequence of consecutive purchases in which every product ID is unique.

```

arr = list(map(int, input().split()))
seen = set()
left = 0
maximum = 0
for right in range(len(arr)):
    while arr[right] in seen:
        seen.remove(arr[left])
        left += 1
    seen.add(arr[right])
    length = right - left + 1
    maximum = max(maximum, length)
print(maximum)

```

Test Case	Input	Expected Output
1	1 2 3 1 4 5	5
2	10 20 30 40 50	5
3	1 1 1 1	1
4	1 2 3 2 4 5 6	5
5	5 4 3 2 1 2 3	5

7. Bank Transaction Analysis A bank stores transaction amounts for a customer's account. A continuous group of transactions may add up to a specific target amount. The auditing system needs to determine how many different continuous transaction groups produce exactly the specified amount.

```
arr = list(map(int, input().split()))
```

```

target = int(input())
prefix_sum = 0
count = 0
seen = {0: 1}
for num in arr:
    prefix_sum += num
    if prefix_sum - target in seen:
        count += seen[prefix_sum - target]
    seen[prefix_sum] = seen.get(prefix_sum, 0) + 1
print(count)

```

Test Case	Transactions	Target	Expected Output
1	1 2 3 2 1	5	2
2	1 1 1 1	2	3
3	1 2 3 4 5	6	2
4	3 -1 2 1 4	5	3
5	2 3 1 2 4	5	2

8. Employee Skill Grouping A company receives a list of employee skill codes represented as strings. Employees having the same set of characters in their skill codes belong to the same skill category, even if the characters appear in a different order. The HR system needs to organize employees into appropriate skill groups.

```

words = input().split()
groups = {}
for word in words:
    key = ''.join(sorted(word))
    if key not in groups:
        groups[key] = []
    groups[key].append(word)
for group in groups.values():
    print(group)

```

Test Case	Input	Expected Groups
1	eat tea tan ate nat bat	[eat, tea, ate] [tan, nat] [bat]
2	abc bca cab dog god	[abc, bca, cab] [dog, god]
3	listen silent enlist hello	[listen, silent, enlist] [hello]
4	cat act tac dog	[cat, act, tac] [dog]
5	a b a ab ba	[a, a] [b] [ab, ba]

9. Network Packet Analysis A network monitoring system receives packet identifiers in chronological order. The system must determine the longest sequence of consecutive packets whose identifiers form a continuous numerical sequence, regardless of their original order in the incoming data

```

arr = list(map(int, input().split()))
numbers = set(arr)
maximum = 0
for num in numbers:

```

```

if num - 1 not in numbers:
    current = num
    length = 1
    while current + 1 in numbers:
        current += 1
        length += 1
    maximum = max(maximum, length)
print(maximum)

```

Test Case	Input	Expected Output
1	100 4 200 1 3 2	4
2	1 2 3 4 5	5
3	10 5 12 3 4 11 2	5
4	1 9 3 10 2 4 20	4
5	5 2 99 1 3 4	5

10. Hospital Appointment Scheduling A hospital receives appointment requests represented by starting and ending times. Some appointments overlap with each other. The scheduling system needs to combine overlapping appointment periods so that the final schedule contains only non-overlapping time ranges.

```

intervals = eval(input())
intervals.sort()
merged = []
for interval in intervals:
    if not merged or interval[0] > merged[-1][1]:
        merged.append(interval)
    else:
        merged[-1][1] = max(merged[-1][1], interval[1])
print(merged)

```

Test Case	Input	Expected Output
1	[[1,3],[2,6],[8,10],[9,12]]	[[1,6],[8,12]]
2	[[1,4],[4,5]]	[[1,5]]
3	[[1,2],[3,4],[5,6]]	[[1,2],[3,4],[5,6]]
4	[[1,10],[2,3],[4,6],[8,9]]	[[1,10]]
5	[[5,7],[1,3],[2,6],[8,10]]	[[1,7],[8,10]]

